HPC 方向技术报告

光线追踪优化与 Mini WorkSteal 运行时

兰州大学超算队 2026 年秋季纳新 • 报告日期 2026-10-27

我完成了光线追踪优化题，并选做了 Mini WorkSteal。光追部分使用 OpenMP 和 BVH，在保持图像结果一致的前提下缩短渲染时间。WorkSteal 则实现一个小型工作窃取运行时，比较任务粒度、初始分配和窃取策略对性能的影响。补充实验和复现方法见文末仓库中的附录。

项目	最终方案	主要结果
光线追踪	03、OpenMP 像素块并行、16 桶 SAH BVH、延后构造命中记录	Render time 中位数 1968.697 → 85.006 秒，耗时比 23.16；六次输出均与参考图逐通道一致
WorkSteal	六类策略分别实现；采用 round_robin、immediate、single	细粒度集中布局中，同一程序关闭与开启窃取的 epoch 时间比为 2.55-5.28；粗粒度下收益不稳定

实验环境为 Intel Core i9-13900HX，Ubuntu 24.04 / WSL1。

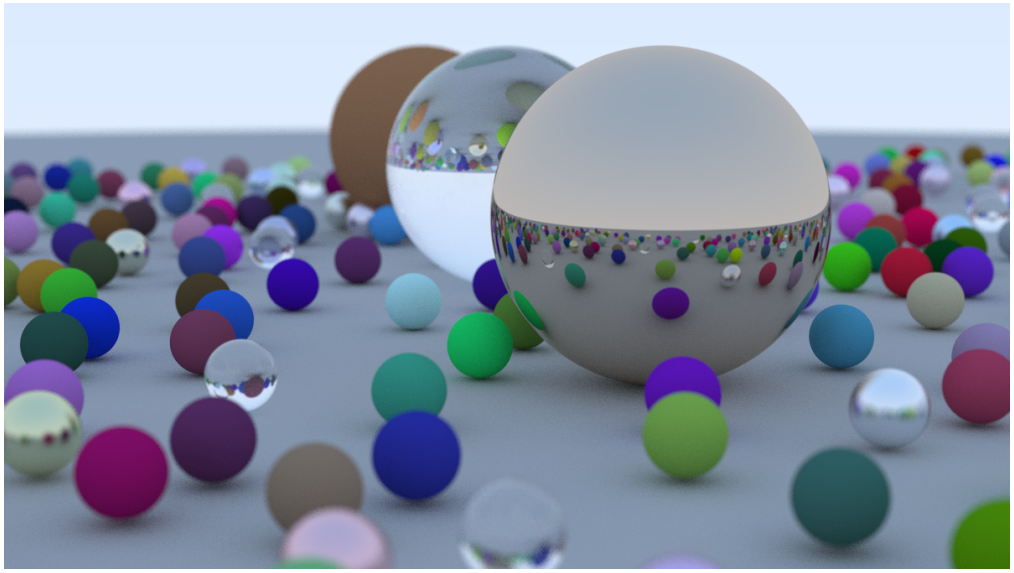


图1：优化后的渲染结果，1200×675，200 spp。

1 光线追踪的分析与实现

1.1 从哪里改起

原版逐像素采样，递归计算光线散射，每次查询场景都线性检查物体。这里有两处可以改：不同像素交给多个线程计算，场景查询用空间结构跳过不可能命中的物体。修改时保持题面规定的采样、散射、天空颜色和随机数规则，也没有改动 main.cpp、common.hpp 和 math/color.*。

1.2 按完整像素并行

我把一个像素的全部样本放在同一个线程里计算。每个像素本来就有独立随机流，这样能保持像素内部的计算顺序和随机数消耗顺序，OpenMP 只改变像素之间的执行顺序。最终按 16×8 像素块动态分配，chunk 为 1，在本机使用 32 个线程。

多个线程如果直接往同一个文件流写入，会互相竞争。我先让各线程写自己的像素缓冲，等并行计算结束，再按原行列顺序输出，额外需要约 18.54 MiB 内存。缓冲分配、计算和文件输出都计入 Render time；BVH 则在场景初始化时建立，耗时另记。

1.3 用 BVH 减少无效求交

BVH 把场景物体分组，用包围盒组织成树。光线没碰到一个盒子，就不用再检查里面的物体。我比较了中位数划分和 16 桶 SAH；SAH 根据子盒面积和物体数量估计求交代价，再选择分组。每个叶子最多放 8 个物体。分桶无法有效划分时，回退到中位数划分。

遍历时先检查入口较近的子树，找到命中后，再用这个距离缩小后续检查范围。BVH 改变了物体的访问顺序，所以除了最近命中，还要处理等距命中和盒边界，保持与原版相同的选择。球体求交仍用 double 和原公式。

1.4 延后构造最终命中记录

原版每发现一个候选命中，就计算位置、法线和材质等信息。如果后面遇到更近的球体，刚算出的记录又会被覆盖。我先比较距离，确定最终命中哪个球体后，再补全记录。这项修改仅对 sphere 类型启用，派生类和其他物体仍使用原接口。

这一版还把该类型的距离计算改成了直接调用。两项修改放在一起测试，因此下文给出的是它们的组合效果。

LTO、PGO 和 SIMD 也做过比较，最终没有保留，相关结果见在线附录A。最终版本仍使用普通 O3 编译。

2 光线追踪的实验结果

2.1 实验设置

按题目要求，图像大小设为 1200×675，每像素采样 200 次，最大深度为 50，seed=42。计时使用程序输出的 Render time，包含渲染和文件输出。原版采用 O2 串行编译，与优化版各运行三次。用 A、B 分别表示原版和优化版，运行顺序为 A1、B1、B2、A2、A3、B3。未额外预热，三次结果全部保留。

配对	原版 O2 / 秒	最终版 / 秒	原版 / 最终版
1	1897.516	84.857	22.36
2	1968.697	85.006	23.16
3	1993.332	85.197	23.40
中位数	1968.697	85.006	23.16

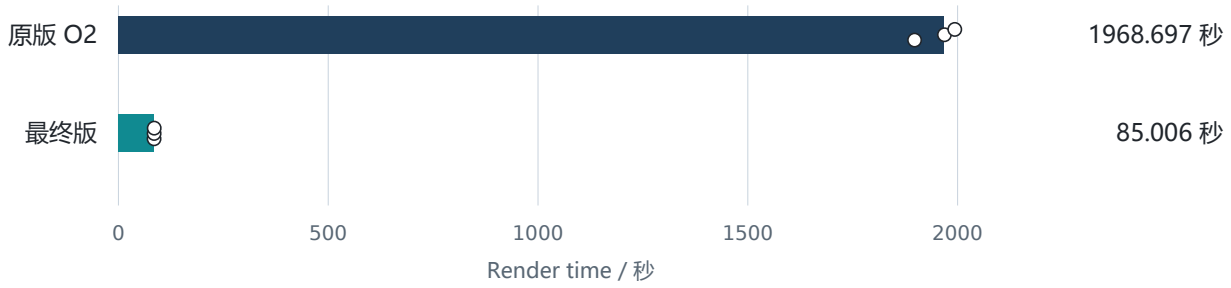


图2：条形表示中位数，圆点表示三次测量结果。优化前后的耗时比为 23.16，这是编译选项、算法和线程配置等修改的共同效果。两组数据都在本机测得，未使用题面中 Xeon 的参考时间。

2.2 图像质量与单项优化效果

使用原始 PPM 文件比较，六次输出都与官方参考及同参数原版逐通道一致，MSE=0，PSNR 为无穷大，满足题目 45 dB 的要求。另外，还比较了修改前后的求交结果和命中记录，检查区间边界、等距命中、特殊方向和派生类型等情况。下表比较 SAH 划分和延后构造命中记录的效果，每个版本测三次，取中位数。

优化项	优化前 / 秒	优化后 / 秒	耗时降低
中位数 BVH → 16 桶 SAH	113.684	96.265	15.32%
即时构造 → 延后构造	97.544	90.667	7.05%

表中两项不是在同一次实验中测得，因此不能把它们的加速比连乘。后续尝试中，我要求新版本在三对比较中都更快，而且最慢的一次也快于旧版本最快的一次，再决定保留。这只是选方案时使用的规则，没有做统计显著性检验。

2.3 耗时波动

同一个优化程序，之前曾测得 17.435 秒，这次对照却在 85 秒左右。测试使用的是日常电脑，没有单独控制 CPU 频率、温度和后台负载，但这么大的差距具体由什么引起，还没有查明。因此，加速比只用本次对照的结果计算。WSL1 下也没有取得可靠的硬件采样，暂时无法判断求交、缓存和同步分别占多少时间。

3 Mini WorkSteal 的实现与测试负载

3.1 固定任务集与策略接口

题目要求实现工作窃取，并比较负载均衡、调度开销和局部性损失。我使用持久线程池和带互斥锁的双端队列，所有任务在执行前创建，不支持运行中递归提交子任务。这样比较不同策略时，总工作量不会改变，也能根据剩余任务数判断是否完成。

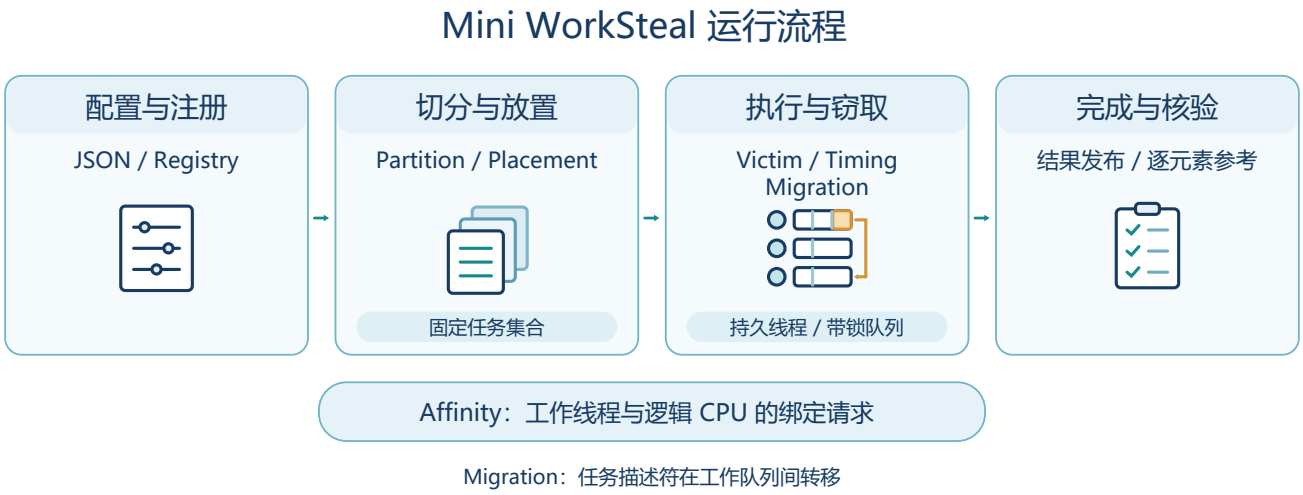


图3：各类策略通过接口组合，线程池负责同步和执行，最后由应用核验结果。Migration 在线程队列间转移任务描述符，Affinity 设置工作线程与逻辑 CPU 的绑定。

策略	职责	代表实现
Partition	任务切分与粒度	fixed_grain、balanced_chunks
Placement	任务初始队列	按 home 或 round_robin 放置
Victim	选择窃取目标	round_robin、seeded_random
Timing	窃取与空闲等待时机	disabled、immediate、bounded_backoff
Migration	单次转移的任务数量	single、capped_half
Affinity	配置线程绑定	unbound、显式逻辑 CPU 列表

每个线程从自己的队尾取任务，空闲时从其他队列的队头窃取。每个任务将结果写入自己的位置，完成后再通知线程池。测试检查任务是否恰好执行一次，以及重复调用、异常恢复和关闭是否正常。另外增加了 test_reverse 策略测试扩展接口：新增策略只需添加实现、注册代码和构建文件，不必改动已有运行时。

3.2 工作负载和对照

测试使用三类负载：均匀负载用于比较任务粒度，偏斜负载用于观察窃取能否分担工作，重复内存访问用于测试局部性。输入由 seed 和元素编号确定，改变 grain（每任务元素数）只改变任务分组。程序会逐元素重新计算来核验结果，小规模输入还与独立的 Python 参考实现比较。

实验还比较了串行、静态队列和集中队列。早先 off 空闲时等待、on 空闲时 yield，时间差夹杂了空闲处理的影响。为此，我增加独立的 disabled_yield 策略，补测时两者都 yield，队列、锁、初始顺序和完成协议保持相同。下文的 epoch 是执行时间，不含建队列和完整核验；app 则包含初始化和核验。

4 任务粒度与窃取收益

4.1 均匀负载中的开销对照

为观察调度路径的成本，我固定均匀负载 N=262144、K=16，在一个工作线程上比较线程池和连续串行循环。三个 grain 都是八路计算内核宽度的整数倍，避免标量尾部差异。每个条件测五次，表中为中位数。

配置	任务数	epoch / ms	相对连续串行
连续串行	单次连续循环	2.3888	1.000
线程池 grain=4096	64	2.3769	0.995
线程池 grain=64	4096	2.5319	1.060
线程池 grain=8	32768	2.8658	1.200

grain 从 4096 减到 8，任务数从 64 增至 32768，epoch 多了 0.4889 毫秒（20.6%）。这是取任务、函数分发和完成通知等综合成本，不能再拆成每把锁的耗时。粗粒度与连续循环接近；单次执行只有几毫秒，细小差距不作排名。

4.2 偏斜布局与不可拆大任务

补测固定同一个程序、16 个工作线程和 K=256，使用两个新输入，每种布局和粒度做 5 对 off/on 比较。两者空闲时都 yield。集中布局的四组实验均为 5/5 对开启窃取更快，epoch 中位数时间比为 2.55–5.28；分散布局为 1.26–1.83。

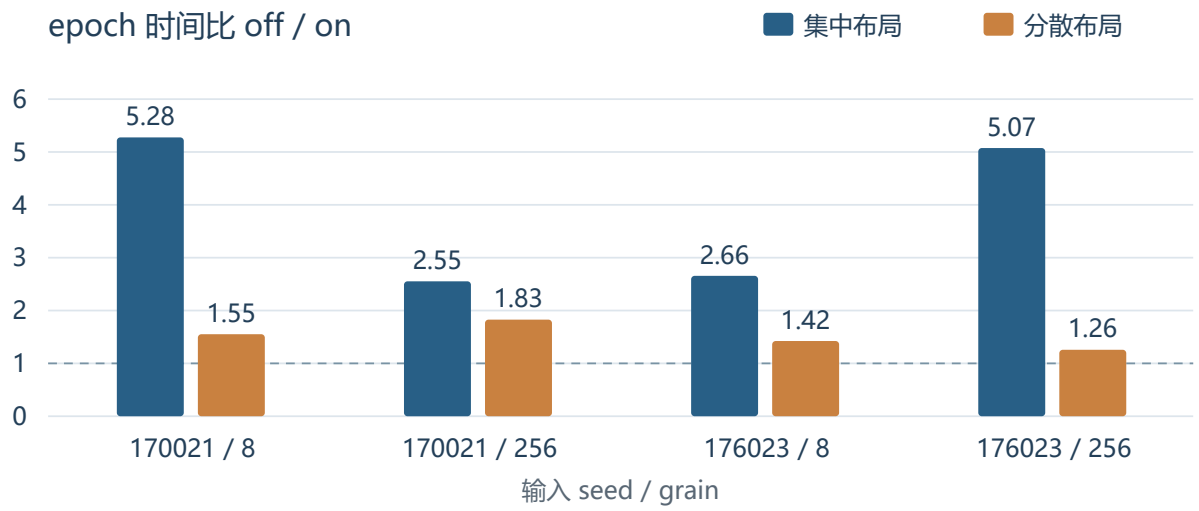


图4：柱高为 off 中位数 / on 中位数，大于 1 表示开启窃取更快，虚线表示两者相等。每个条件测五次，输入规模分别为 65536 和 98304。

另一次粗粒度实验中，grain=16384 时只形成 4 个和 6 个任务，最大任务占总计算量的 62.5% 和 58.33%，窃取没有稳定收益。已开始的任务不能再拆开，换 victim 也无法分担这个大任务。计算量按 mix 迭代次数估计，并不直接等于实际耗时；app 的改善也不能用 epoch 的时间比代替。

5 局部性与最终策略选择

5.1 预读线程与局部性

系统显示 LLC 约为 36 MiB。补测固定执行线程，比较由它自己或另一个线程预读数据。64 个任务的总数据分别为 512 KiB、4.5 MiB 和 288 MiB，每任务为 8 KiB、72 KiB 和 4.5 MiB，每个条件测五次。下表耗时比为“另一线程预读 / 执行线程预读”。

总数据	执行0 耗时比	执行1 耗时比	成功窃取次数
512 KiB	1.021	0.956	46
4.5 MiB	1.007	1.002	47
288 MiB	1.019	0.974	48

最后一列来自另外的四线程动态诊断，三种规模分别有 46、47、48 个任务由非 home 线程执行，确认实际发生了窃取。预读对照的两个方向仍没有一致的变慢趋势。WSL1 下未确认物理缓存映射，也没有可靠 PMU，因此不能把预读差异直接当作纯缓存损失，更不能据此说局部性没有代价。三个效应也不能作为互不影响的分量相加。

为覆盖单任务远大于 LLC 的一端，我另测每任务 288 MiB、共四任务的情况。动态诊断有 3/4 个任务由其他线程执行；固定执行线程后的两个预读耗时比分别为 1.009 和 0.988，仍没有一致的变慢趋势。

5.2 一次窃取多个任务

一次多取几个任务，可能减少反复窃取的开销。我固定程序、线程数、粒度和输入，比较每次取一个任务的 single，以及最多取 4、16、64 个任务的批量方案。cap4 初测较快，因此再换两个输入，每个输入做 7 对比较。下表是集中布局的结果。

输入 seed	epoch single→cap4 ms	更快对数	app single→cap4 ms
116009	3.7183 → 3.6328	3/7	31.6320 → 30.2063
122011	5.0438 → 5.1210	4/7	38.1532 → 41.2096

cap4 在筛选时让 epoch 下降约 22.7%，但换输入后只有 3/7、4/7 对更快，第二组 app 中位数还增加了 8.01%。虽然诊断显示成功窃取次数减少、成本分配更均衡，但迁移的描述符总量也增加了。收益不够稳定，最后仍用 single。

5.3 最终配置

最终采用 fixed_grain、round_robin victim、immediate 和 single，并指定线程绑定的逻辑 CPU。placement 与 layout 决定初始任务分配，线程数和粒度按负载调整，具体配置见在线附录B。

完整核验并行化和内存内核调整的结果见在线附录A。这些修改改善的是应用中的其他环节，不能都算作工作窃取带来的加速。

源码与补充实验

源码、构建步骤、补充实验和运行日志见：

https://github.com/daicy2005/lzuhpc-2026-hpc/tree/36e6a13f386f9aa3465c2515a5c32bf2bb34a783

AI 辅助包括资料搜集、备选方案梳理、部分代码编写和数据整理。